A COMPARISON OF POOLING METHODS ON LSTM MODELS FOR RARE ACOUSTIC EVENT CLASSIFICATION

Chieh-Chi Kao[1], *Ming Sun*[1], *Weiran Wang*[2*], *Chao Wang*[1]

[1]Amazon.com Inc. [2]Salesforce Research

{chiehchi,mingsun}@amazon.com weiran.wang@salesforce.com wngcha@amazon.com

ABSTRACT

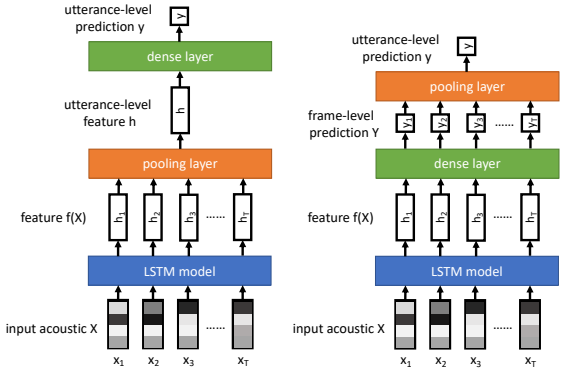
Acoustic event classification (AEC) and acoustic event detection (AED) refer to the task of detecting whether specific target events occur in audios. As long short-term memory (LSTM) leads to state-of-the-art results in various speech related tasks, it is employed as a popular solution for AEC as well. This paper focuses on investigating the dynamics of LSTM model on AEC tasks. It includes a detailed analysis on LSTM memory retaining, and a benchmarking of nine different pooling methods on LSTM models using 1.7M generated mixture clips of multiple events with different signal-to-noise ratios. This paper focuses on understanding: 1) utterance-level classification accuracy; 2) sensitivity to event position within an utterance. The analysis is done on the dataset for the detection of rare sound events from DCASE 2017 Challenge. We find max pooling on the prediction level to perform the best among the nine pooling approaches in terms of classification accuracy and insensitivity to event position within an utterance. To authors' best knowledge, this is the first kind of such work focused on LSTM dynamics for AEC tasks.

Index Terms— Long short-term memory (LSTM), acoustic event classification and detection, pooling functions

1. INTRODUCTION

Acoustic event classification (AEC) and acoustic event detection (AED) refer to the task of detecting whether specific target events occur in audios. It is of interest in many real-world scenarios, e.g. traffic monitoring [1], surveillance [2, 3], etc. In recent years, with deep learning based solutions burgeoning in various areas including speech and language processing [4, 5], and computer vision [6], there has been intensive research on improving AED performance via neural networks. For instance, several new neural network architectures [7, 8, 9] were proposed for AED.

As long short-term memory (LSTM) [10] leads to state-of-the-art results in various speech related tasks, e.g. automatic speech recognition [4], keyword spotting [11], speaker identification [12], whisper detection [13], it is employed as a popular solution for AEC as well [14, 15, 16, 17, 18, 19, 20], typically combined with convolutional neural networks (CNNs) [21]. To run applications mentioned above on mobile devices or smart speakers, a model with small memory footprint is required. LSTM models have much less number of parameters than CNN models while reasonable performance maintained. Besides, LSTM models can operate in a streaming mode with a ring buffer, which has the benefit of low latency. We are interested in understanding the memory dynamics for LSTM based AEC models, i.e. for how long LSTM retains memory of happening acoustic events, as well as how to improve memory retaining for specific pooling methods. Recent research investigates memory dynamics and control in recurrent neural networks (RNNs) including LSTM [22]. There are comparisons on different pooling functions for AEC/AED [9], and spatio-temporal attention pooling proposed for audio scene classification [23].



(a) Pooling on feature (b) Pooling on prediction

Fig. 1: Two categories of pooling methods to generate utterance-level prediction for LSTM models.

Wang et al. [9] did a thorough analysis theoretically and experimentally of five pooling functions on prediction. The analysis was done for multiple instance learning framework on AED with weak labeling, whose goal is to detect and localize events at the same time. Their experiments were done on DCASE 2017 task 4 [24]: weakly supervised AED for smart cars. This dataset employs a subset of AudioSet [25], which includes 10-second clips containing 17 sound events from two categories: "Warning" and "Vehicle". Although audio tagging, which is similar to our utterance-level classification task, is covered in the experiments in [9], there are two fundamental differences between this work and theirs. 1) **Different characteristics of events**: Our work focuses on rare events, and we conducted experiments on DCASE 2017 task 2 [24]: detection of rare sound events. The averaged length of target events in the test set is shorter than 2 seconds [26], and these events only occupy a very small portion of the whole utterance (30 seconds). On the contrary, for events in DCASE 2017 task 4 (e.g. "Ambulance (siren)", "Car passing by", "Train", etc.), the sound may cover the whole 10-second clip. 2) **Different focuses of analysis**: In [9], the analysis focuses on the effect of pooling functions on both event localization and classification for weak labeling. Due to the requirement of event localization, only pooling functions on prediction were discussed in [9]. Our work analyzes the effect of pooling functions on LSTM based AEC mod-

*This work was done while the author was at Amazon.

Table 1: Pooling methods on feature.

Pooling method	Function
Last frame	$\boldsymbol{h} = \boldsymbol{h}_T$
Attention	$\boldsymbol{h} = \sum_t a_t \boldsymbol{h}_t$
Max pooling	$\boldsymbol{h}[n] = \max_t \boldsymbol{h}_t[n]$
Average pooling	$\boldsymbol{h} = \frac{1}{T} \sum_t \boldsymbol{h}_t$

Table 2: Pooling methods on prediction.

Pooling method	Function
Max pooling	$y = \max_t y_t$
Average pooling	$y = \frac{1}{T} \sum_t y_t$
Linear softmax	$y = \frac{\sum_t y_t^2}{\sum_t y_t}$
Exponential softmax	$y = \frac{\sum_t y_t \exp(y_t)}{\sum_t \exp(y_t)}$
Attention	$y = \frac{\sum_t y_t w_t}{\sum_t w_t}$

els focusing on utterance-level classification. Experiments are designed for understanding the memory dynamics for LSTM models, and looking for solutions to mitigate the sensitivity to event positions. Moreover, we further discuss four pooling functions on the feature side, which are not covered in [9].

In this paper, we investigate the dynamics of LSTM memory on AEC tasks, including an analysis on LSTM memory retaining, and a benchmarking for impacts of different pooling approaches on LSTM memory dynamics and AEC accuracies, using 1.7M synthesized clips (across 3 event types and 3 SNRs). We also show that bi-directional LSTM can mitigate the sensitivity to event positions for certain pooling methods.

2. POOLING FUNCTIONS FOR LSTM MODELS

Denote an input utterance by $\boldsymbol{X}$=[$\boldsymbol{x}_1$,...,$\boldsymbol{x}_T$] where $\boldsymbol{x}_i \in \mathbb{R}^d$ contains the audio features for the i-th frame. Since our task is to detect whether the specific target event occurs in the utterance, a binary label $\hat{y}$ which indicates if an event occurs ($\hat{y}$ = 1) or not ($\hat{y}$ = 0) is given for each utterance. Our goal is to make accurate predictions at utterance-level for rare acoustic event classification.

Our model uses an LSTM model f, which may contain one or multiple LSTM layers, to extract nonlinear features from $\boldsymbol{X}$. This gives a representation feature containing temporal information as: $f(\boldsymbol{X}) = [\boldsymbol{h}_1, ..., \boldsymbol{h}_T] \in \mathbb{R}^{N \times T}$. Given this feature $f(\boldsymbol{X})$, different pooling methods can be applied to generate the utterance-level prediction y. We can categorize pooling methods into two types: *pooling on feature* and *pooling on prediction*.

2.1. Pooling on feature

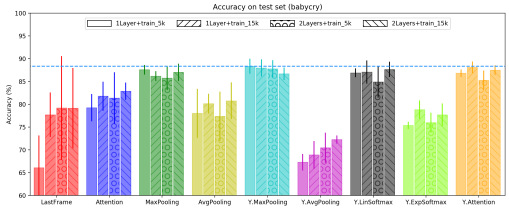
Given the feature $f(\boldsymbol{X})$ generated by an LSTM model, we can aggregate the frame-level features to generate the utterance-level feature $\boldsymbol{h}$. The utterance-level feature is then fed into a dense layer with sigmoid activation to generate the utterance-level prediction y. We list the definition of four feature-level pooling functions in Table 1.

Using the feature of the last frame in the utterance ($\boldsymbol{h}_T$) is the simplest way to generate the utterance-level feature. Since LSTM models have the ability to capture contextual information, we can directly use the feature of the last frame to represent the whole utterance. However, this naïve method may suffer from long-term memory loss. Li et al. [27] showed that LSTM models could only memorize less than 1,000 steps. Our experiments shown in Sec. 4.1 also have demonstrated that this method has the forgetting issue on AEC.

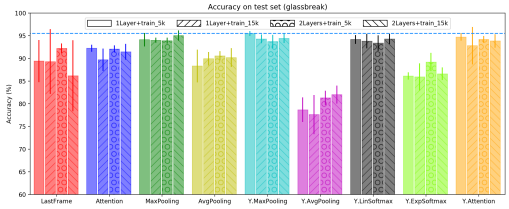
The attention pooling function uses attention weights (a_t) to combine frame-level features to form the utterance-level feature. We use the same architecture as proposed in [8] to generate attention weights. The weights for each frame (a_t) are learned with a dense layer, whose weights are shared with the final dense layer generating the utterance-level prediction. This design encourages the attention to be peaked at frames containing target events, and be suppressed at the non-event frames. In this work, there is no frame-level supervision for attention, which is slightly different from [8].

The max pooling function takes the largest value at each feature channel across all time frames. Intuitively, it should work well for rare event detection since the length of target event is short compared to the utterance length. Once the model detects the event at certain frames, these strong responses will be kept through max pooling without suffering from the forgetting issue. Our experiments also have shown that this is one of the best performing pooling methods.

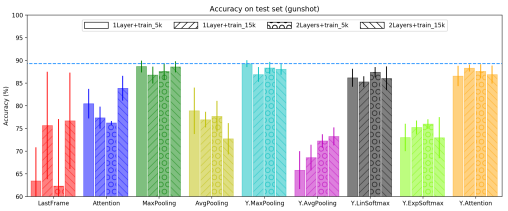
The average pooling function simply assigns the same weight ($\frac{1}{T}$) to all frames. This setup is not suitable for rare AEC since the number of frames with events is small, which makes the utterance-level feature difficult to represent the events.



(a) Babycry



(b) Glassbreak



(c) Gunshot

Fig. 2: Accuracies on the test set for LSTM models with different pooling methods. Each bar represents an average of five trials, and the error bar is the sample standard deviation of five trials. Note that names starting with "*Y.*" are pooling methods on prediction, and the rest are pooling on feature. The horizontal dashed line is the best accuracy out of all setups.

2.2. Pooling on prediction

Given the feature $\boldsymbol{x}_t$ generated by LSTM model at each frame t, we can generate the frame-level prediction y_t. These frame-level predictions $\boldsymbol{Y} = [y_1, ..., y_T]$ are fed into a pooling layer to generate the utterance-level prediction y. We list the definition of five prediction-level pooling functions covered in [9] in Table 2. Max and average pooling generate the utterance-level prediction by simply taking the

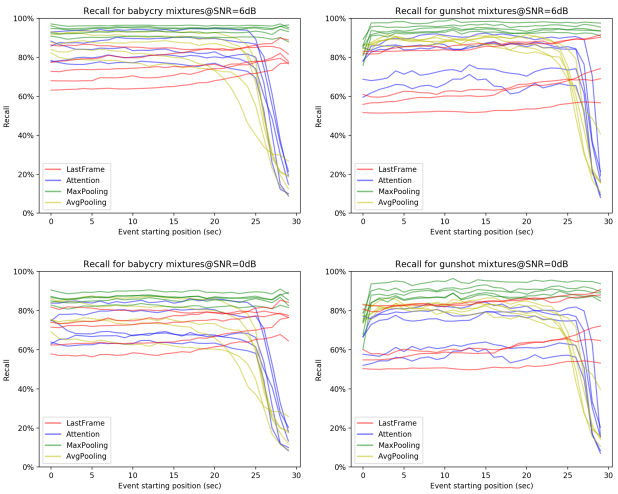


Fig. 3: Recall rate for methods of pooling on feature for 'babycry' abd 'gunshot'. Mixtures used in this evaluation are generated by placing event segments at different event positions. Rows from top to bottom are mixtures with EBR of 6dB, 0dB respectively.

max and mean of frame-level predictions across all time frames.

The two softmax pooling functions are weighted sum of frame-level predictions, where the weights for linear softmax is the frame-level prediction itself (y_t), and the weights for exponential softmax is the exponential of frame-level prediction ($\exp(y_t)$). These softmax pooling functions give the frames with high prediction values more influence on the utterance-level prediction.

The attention pooling function uses attention weights (w_t) learned from a dedicated dense layer within the network, as proposed in [9]. Note that the attention weights here (w_t) are different from the one used in attention pooling on feature (a_t). Unlike w_t, we don't have an extra dense layer to generate the attention weights for a_t, which is generated by sharing the weights with the dense layer generating utterance prediction y as explained in [8].

3. EXPERIMENTAL SETUP

3.1. Dataset

We tested nine pooling methods on LSTM models using the dataset provided by DCASE 2017 Challenge task 2 [24]: "Detection of rare sound events." The task data consist of three isolated target events (baby crying, glass breaking, and gunshot) downloaded from freesound.org, and 30-second background sound clips including fifteen different audio scenes (bus, cafe, home, library, etc.) from TUT Acoustic Scenes 2016 dataset [28]. Rare events are relatively short (averaged length shorter than 2 seconds in the test set) compared with the background clips. A synthesizer provided by the challenge organizer is used to generate mixtures of target events and background sound clips at random onset time. For each target event, we generate 5,000 or 15,000 training samples with event-to-background ratios (EBR) of -6, 0, 6dB. These EBRs are chosen to match the data distribution of mixture clips provided in the dev/test sets. For each generated training set, half of the utterances contain the target event, and the other half don't. For the development and test sets, there are 500 mixtures provided by the challenge organizer in each set, and the ratio of containing event is 0.5 as well. The evaluation metric we used for utterance-level binary classification is accuracy, and the threshold is set to 0.5 to all models in this work.

The synthesized mixtures are 30-second monaural audio with 44,100 Hz and 24 bits. We use log filter bank energies (LFBEs) as the acoustic features in this work. We decompose each mixture clip into a sequence of 25 ms frames with a 10 ms shift. 64 dimensional LFBEs are calculated for each frame, and we aggregate the LFBEs from all frames to generate the input spectrogram (3,000×64).

3.2. Training Setup

We use models consist of one or two uni-directional LSTM layers, followed by different pooling methods to generate the utterance-level prediction. We set the number of units to 100 for all LSTM layers. We use the loss of the development set as the criterion for model selection. Adaptive momentum (ADAM) [29] is used as the optimizer and the initial learning rate is set to 0.001. The size of mini-batch is set to 200 for the training set of 5k samples, and 600 for the training set of 15k samples. One exception is that for attention pooling on feature, we set it to 500 for model with one LSTM layer and 400 for model with two LSTM layers on the training set of 15k samples due to the memory constraints on the GPU. We use Keras with Tensorflow backend on Tesla K80 GPUs to conduct the experiments.

3.3. Dataset for Testing Sensitivity to Event Positions

In order to check the sensitivity to event positions for different pooling methods, we need a dataset containing mixtures with the target event placed at different timestamps. We use the source data in the test set to generate such mixtures. We first randomly selected 100 30-second background clips out of 387 clips from the test set. For each event segment, we generate the mixtures by placing it at different positions $t \in [0, 1, ..., 29]$, and mix it with the background clip at three EBR of -6, 0, 6dB. Depending on the length of event segment, it may generate up to 30 mixtures for each pair of event segment and background clip. In the test set, there are 61, 58, and 76 segments for 'babycry', 'glassbreak', and 'gunshot' respectively. We have generated roughly 1.76M 30-second mixtures ((61+58+76)×100×30×3) for benchmarking the sensitivity of pooling methods to event positions. We use the recall rate on mixtures as the evaluation metric since all mixtures are positive samples for the classification task.

4. EXPERIMENTAL RESULTS

For each type of event, we first experiment nine pooling methods on two models (1 and 2 LSTM layers) and two sizes of the training set (5k and 15k). The accuracies on the test set are shown in Fig. 2. For architectures explored here with the best pooling (Y.MaxPooling), we observed that neither increasing the number of mixtures in the training set nor increasing the model complexity can further improve the accuracy. This shows that models with a single LSTM layer have enough model complexity for DCASE 2017 task 2 dataset. Therefore, we conduct our further analysis of different pooling methods on models with a single LSTM layer trained on the 5k dataset.

As shown in Fig. 2, max pooling on prediction outperforms other eight pooling methods across three event types. If we take the average of accuracies (1Layer+train_5k) shown in Fig. 2 over three types of events, we can find the top-4 pooling methods ranked as following: 1) Y.MaxPooling (91.03%), 2) MaxPooling (90.11%), 3) Y.Attention (89.36%), 4) Y.LinSoftmax (89.07%). This result is not consistent with the finding in [9], where max pooling on prediction performs the worst for audio tagging. We hypothesize that

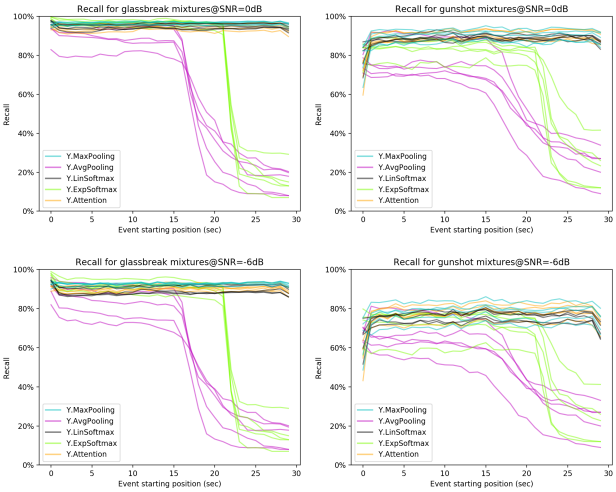


Fig. 4: Recall rate for methods of pooling on prediction for 'babycry' and 'gunshot'. Mixtures used in this evaluation are generated by placing event segments at different event positions. Rows from top to down are mixtures with EBR of 0dB, -6dB respectively.

this inconsistency is due to the different characteristics between two datasets, and more details are covered in Sec. 4.1.

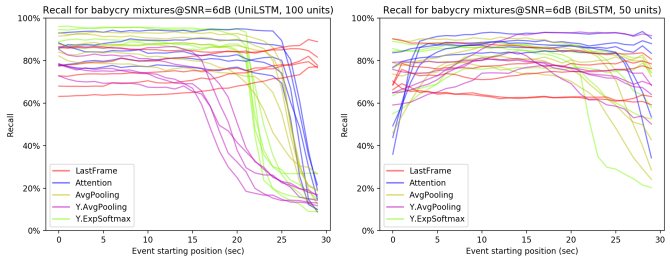
4.1. Dynamics of LSTM Models on AEC

If the LSTM model is able to retain long-term memory over thousands of steps, it should be able to detect the event no matter where the event occurs within the 30-second utterance. We investigate the dynamics of LSTM memory on AEC tasks by testing the models on mixtures with event segments placed at different timestamps. For each model, we evaluate the recall rate on mixtures with an event segment placed at time t, and observe how the recall rate changes with respect to t. Curves for pooling on feature methods (Fig. 3) and pooling on prediction methods (Fig. 4) were calculated on 1.7M mixtures synthesized with the setup described in Sec.3.3.

Memory retaining Li et al. [27] showed that LSTM can only keep a mid-range memory (about 500-1,000 time steps). To check if LSTM models have a similar memory forgetting issue on AEC, we can look at the red curves of 'LastFrame' in Fig. 3. For 'babycry' and 'gunshot' events, we can see the recall rate goes up with the increase of t, which shows that the LSTM model has higher chances to forget events happened close to the beginning of utterance. This trend is universal across different EBR settings. These results suggest that it is essential to choose a pooling method for LSTM models suitable for rare AEC.

Sensitivity to event positions From Fig. 3 and Fig. 4, we can tell that the top 4 performing methods (Y.MaxPooling, MaxPooling, Y.Attention, and Y.LinSoftmax) are not sensitive to event positions. Overall, the recall rates do not change a lot in respect of t. Interestingly, there is a huge drop in recall rate when 'gunshot' events are placed at $t = 0$ for the top-4 methods. We suspect that it is because of an issue of annotation for the onset time. Models learned to identify 'gunshot' sounds by detecting the sharp change in the amplitude of sounds. The onset time of some 'gunshot' events are labeled after the actual onset time, which means that there is no huge amplitude change in the event segment. When these segments get placed at $t = 0$, models are not able to detect them correctly.

As the magenta curves shown in Fig. 4, the top performing pooling method for audio tagging (Y.AvgPooling) reported in [9] is very sensitive to event position. If the event is placed at after half of the mixture ($t >= 15$), the recall rate decreases quickly with respect to t. This verifies our hypothesis that different characteristics of datasets lead to findings inconsistent with [9]. In [9], the event length is much longer than the rare events in our setup and it covers a huge part of an utterance, which means the onset time of the event is close to the beginning of the utterance. Y.AvgPooling works for that case since sensitivity to event position is not an issue anymore, and it can also avoid some noises brought by using max pooling. On the contrary, Y.AvgPooling is not suitable for rare AEC due to that the event length is short than 2 seconds, and the sensitivity to event position matters a lot.



(a) Uni(100 units) (b) Bi(50 units)

Fig. 5: We applied 5 pooling methods sensitive to event positions on uni-directional (Uni) and bi-directional (Bi) LSTM models. Recall rates are calculated on mixture clips of 'babycry' with 6dB SNR.

Mitigation of sensitivity After observing the sensitivity to event positions for certain pooling methods, we are looking for a solution to mitigate this effect. A straight forward idea is to apply these pooling methods to a bi-directional LSTM model. We have trained a set of bi-directional LSTM models with 50 units in each direction. This setup generates feature maps with 100 units at each frame, which is the same as the default model (100 units uni-directional LSTM model). For each pooling method, we trained 5 models to reduce the randomness during the training. We tested these models on mixture clips of 'babycry' event with 6dB SNR and the recall rates are shown in Fig. 5. As shown in Fig. 5b, the sensitivity to event positions is reduced significantly by using bi-directional LSTM. Compared with Fig. 5a, the worst recall rate of five trials has improved from 10% to 50% for Y.AvgPooling, from 10% to 35% for Attention, from 10% to 25% for AvgPooling.

5. CONCLUSION

We have benchmarked nine different pooling methods for LSTM AEC models on task 2 of the DCASE 2017 challenge, with 1.7M mixtures generated to evaluate utterance-level classification accuracy and sensitivity to event positions. We found that max pooling on the prediction level (Y.MaxPooling) is the best performing method in terms of classification accuracy, and it is also robust to event positions. This observation is different from the finding in [9], where max pooling on prediction is the worst performing method in audio tagging task. We hypothesize that this inconsistency is due to the different characteristics between two datasets as discussed in Sec. 4.1. We also explored using bi-directional LSTM models to mitigate the sensitivity issue for certain pooling methods. To authors' best knowledge, this is the first work focusing on LSTM memory dynamics for AEC tasks.

6. REFERENCES

[1] Shuangwu Chen, ZP Sun, and B Bridge, "Automatic traffic monitoring by intelligent sound detection," in *Proceedings of IEEE Conference on Intelligent Transportation Systems*, 1997, pp. 171–176.

[2] Marco Cristani, Manuele Bicego, and Vittorio Murino, "Audio-visual event recognition in surveillance video sequences," *IEEE Transactions on Multimedia*, vol. 9, no. 2, pp. 257–267, 2007.

[3] Giuseppe Valenzise, Luigi Gerosa, Marco Tagliasacchi, Fabio Antonacci, and Augusto Sarti, "Scream and gunshot detection and localization for audio-surveillance systems," in *IEEE Conference on Advanced Video and Signal Based Surveillance*, 2007, pp. 21–26.

[4] Geoffrey Hinton, Li Deng, Dong Yu, George Dahl, Abdelrahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Brian Kingsbury, et al., "Deep neural networks for acoustic modeling in speech recognition," *IEEE Signal processing magazine*, vol. 29, 2012.

[5] Martin Sundermeyer, Ralf Schlter, and Hermann Ney, "Lstm neural networks for language modeling," *Interspeech*, 2012.

[6] Wonmin Byeon, Thomas M. Breuel, Federico Raue, and Marcus Liwicki, "Scene labeling with lstm recurrent neural networks," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015.

[7] Chieh-Chi Kao, Weiran Wang, Ming Sun, and Chao Wang, "R-crnn: Region-based convolutional recurrent neural network for audio event detection," *Interspeech*, 2018.

[8] Weiran Wang, Chieh-Chi Kao, and Chao Wang, "A simple model for detection of rare sound events," *Interspeech*, 2018.

[9] Yun Wang, Juncheng Li, and Florian Metze, "A comparison of five multiple instance learning pooling functions for sound event detection with weak labeling," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019.

[10] Sepp Hochreiter and Jürgen Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] Sercan O. Arik, Markus Kliegl, Rewon Child, Joel Hestness, Andrew Gibiansky, Chris Fougner, Ryan Prenger, and Adam Coates, "Convolutional recurrent neural networks for small-footprint keyword spotting," in *Interspeech*, 2017.

[12] Jimmy Ren, Yongtao Hu, Yu-Wing Tai, Chuan Wang, Li Xu, Wenxiu Sun, and Qiong Yan, "Look, listen and learn – a multimodal lstm for speaker identification," in *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[13] Zeynab Raeesy, Kellen Gillespie, Chengyuan Ma, Thomas Drugman, Jiacheng Gu, Roland Maas, Ariya Rastrow, and Björn Hoffmeister, "Lstm-based whisper detection," in *IEEE Spoken Language Technology Workshop (SLT)*, 2018, pp. 139–144.

[14] T. Hayashi, S. Watanabe, T. Toda, T. Hori, J. Le Roux, and K. Takeda, "Duration-controlled lstm for polyphonic sound event detection," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 11, pp. 2059–2070, Nov 2017.

[15] Yun Wang, Leonardo Neves, and Florian Metze, "Audio-based multimedia event detection using deep recurrent neural networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016.

[16] G. Parascandolo, H. Huttunen, and T. Virtanen, "Recurrent neural networks for polyphonic sound event detection in real life recordings," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016.

[17] Jinxi Guo, Ning Xu, Li-Jia Li, and Abeer Alwan, "Attention based cldnns for short-duration acoustic scene classification," *Interspeech*, 2017.

[18] Tang Qingming, Ming Sun, Chieh-Chi Kao, Viktor Rozgic, and Chao Wang, "Hierarchical residual-pyramidal model for large context based media presence detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019.

[19] Bowen Shi, Ming Sun, Chieh-Chi Kao, Viktor Rozgic, Spyros Matsoukas, and Chao Wang, "Compression of acoustic event detection models with low-rank matrix factorization and quantization training," *NeurIPS workshop on Compact Deep Neural Networks with industrial applications*, 2018.

[20] Bowen Shi, Ming Sun, Chieh-Chi Kao, Viktor Rozgic, Spyros Matsoukas, and Chao Wang, "Compression of acoustic event detection models with quantized distillation," *Interspeech*, 2019.

[21] Hyungui Lim, Jeongsoo Park, and Yoonchang Han, "Rare sound event detection using 1D convolutional recurrent neural networks," Tech. Rep., DCASE2017 Challenge.

[22] Doron Haviv, Alexander Rivkind, and Omri Barak, "Understanding and controlling memory in recurrent neural networks," *ICML*, 2019.

[23] Huy Phan, Oliver Y. Chen, Lam Pham, Philipp Koch, and Maarten De Vos, "Spatio-temporal attention pooling for audio scene classification," *Interspeech*, 2019.

[24] A. Mesaros, T. Heittola, A. Diment, B. Elizalde, A. Shah, E. Vincent, B. Raj, and T. Virtanen, "DCASE 2017 challenge setup: Tasks, datasets and baseline system," in *Proceedings of the Detection and Classification of Acoustic Scenes and Events Workshop (DCASE)*, pp. 85–92.

[25] Jort F. Gemmeke, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter, "Audio set: An ontology and human-labeled dataset for audio events," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 776–780.

[26] Jun Wang and Shengchen Li, "Multi-frame concatenation for detection of rare sound events based on deep neural network," *DCASE 2017 Challenge*, 2017.

[27] Shuai Li, Wanqing Li, Chris Cook, Ce Zhu, and Yanbo Gao, "Independently recurrent neural network (indrnn): Building a longer and deeper rnn," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.

[28] A. Mesaros, T. Heittola, and T. Virtanen, "Tut database for acoustic scene classification and sound event detection," in *2016 24th European Signal Processing Conference (EUSIPCO)*, Aug 2016, pp. 1128–1132.

[29] Diederik P. Kingma and Jimmy Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014.